

SQL

Introduzione

Prof. Ionel Eduard Stan
2025/2026

Riadattamento delle slides del prof. Simone Melzi

CAP 4

BASI DI DATI

ATZENI CERI PARABOSCHI TORLONE

Introduzione a SQL

Originariamente: Structured Query Language

Ora: nome proprio del Linguaggio

Il linguaggio SQL è un linguaggio per la **definizione** e la **manipolazione** dei dati in database relazionali (laboratorio IBM a San Josè, California), adottato da molti DBMS.

- Dal 1983 è uno standard “de facto”
- Dal 1986 prima versione ufficiale
- Poi altre versioni SQL-89, SQL-2, SQL-3 ...

Per la Storia di SQL si rimanda al **Libro** (Capitolo 4.1)

Funzionalità di SQL

SQL è un linguaggio con varie funzionalità che contiene:

- **DDL (Data Definition Language):**
Operazioni di definizione e modifica di schemi
- **DML (Data Manipulation Language):**
Operazioni di interrogazione e di aggiornamento

SQL-2

Noi faremo riferimento principalmente a SQL-2

SQL-2 è ricco e complesso, nessun sistema commerciale lo implementa in maniera completa (3 livelli di conformità):

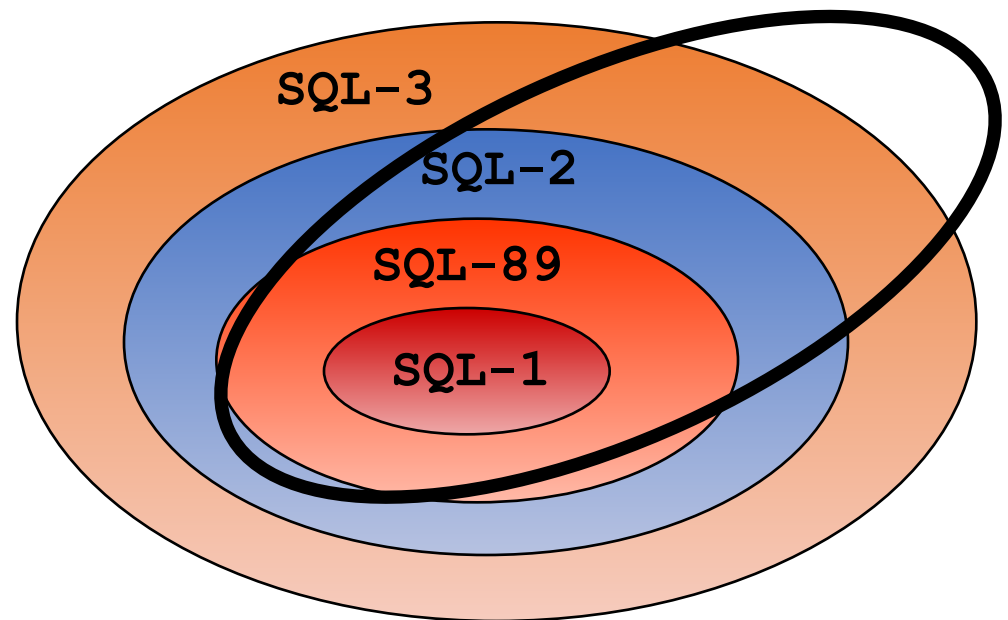
- **Entry**: molto simile a SQL - 89
 - **Intermediate**: versione che soddisfa le esigenze del mercato
 - **Full**: versione completa anche delle funzioni avanzate che non sono realizzate in alcun DBMS
-
- La maggior parte dei database è conforme solo all'Entry Level.
 - Le differenze sono nei dettagli (<http://troels.arvin.dk/db/rdbms/>)

Implementazioni di SQL

Alcuni DBMS

- ORACLE
- DB2 (IBM)
- Access (Microsoft)
- MSSQL server (Microsoft)
- Informix
- Mysql
- Interbase (Borland / O.S.)
- FireBird (Open Source)
- ...

**Un DBMS
tipico**



SQL e ALGEBRA Relazionale

- SQL è **relazionalmente completo**: ogni espressione dell'algebra relazionale può essere tradotta in SQL
- SQL adotta la logica a 3 valori (T, F, U) dell'algebra relazionale (per poter gestire i valori nulli U = Unknown)
- Il modello dei dati di SQL è basato su **tabelle** anziché **relazioni** (Possono essere presenti righe duplicate)
- SQL è computazionalmente completo, ha istruzioni di controllo

SQL – Istruzioni Principali DDL

Operazioni di definizione schema e modifica	CREATE	Definisce database, tabelle, domini, viste, vincoli, autorizzazioni
	ALTER	Modifica attributi e vincoli
	DROP	Elimina database e tabelle

SQL – Istruzioni Principali DML

Operazioni di interrogazione { **SELECT** Formula query come quelle dell'AR o richieste più elaborate

Operazioni di aggiornamento { **INSERT** Inserisce nuove tuple nelle tabelle
DELETE Elimina tuple nelle tabelle
UPDATE Modifica tuple

Quest'ultime possono basarsi sul risultato di una query

SQL è dichiarativo

- SQL è principalmente dichiarativo
- **Non possiamo scegliere l'ordine** in cui avvengono le operazioni
- È necessario attenersi alla **struttura sintattica delle istruzioni**

Notazione SQL

TERMINI DEL LINGUAGGIO in maiuscolo

termini variabili (specificati dall'utente) in minuscolo

< x > usate per isolare un termine x

[x] indicano che termine x è opzionale

{ x } indicano che termine x può essere ripetuto 0 volte o un numero arbitrario di volte

| separa opzioni alternative

Es: x | y indica che termini x e y sono alternativi

Le parentesi tonde () appartengono al linguaggio SQL

SQL un primo esempio di query

```
SELECT ListaAttributi  
FROM ListaTabella  
[WHERE Condizione]
```

Elementi
Obbligatori

Se vogliamo mettere
condizioni alla nostra
selezione dobbiamo
specificarlo qui

SQL Data Definition Language

Prof. Ionel Eduard Stan
2025/2026

Riadattamento delle slides del prof. Simone Melzi

CAP 4

BASI DI DATI

ATZENI CERI PARABOSCHI TORLONE

Definizione di schemi

Uno **schema** di base di dati è una collezione di oggetti:
domini, tabelle, asserzioni, viste, privilegi...

uno schema è definito dalla sintassi:

```
CREATE SCHEMA
```

```
[NomeSchema]
```

```
[ [ authorization ] Autorizzazione ]
```

```
{ DefinizioneElementoSchema }
```

se omissso è nome
utente che ha
lanciato il comando

Termine del linguaggio

proprietario dello
schema (utente
che lo ha definito)

Termine variabile

Definizione di schemi

```
CREATE SCHEMA  
    [NomeSchema]  
    [ [ authorization ] Autorizzazione ]  
    { DefinizioneElementoSchema }
```

Dopo il comando **CREATE SCHEMA** compaiono le definizioni dei vari elementi.

Non è necessario che la definizione di tutti gli elementi avvenga contemporaneamente alla creazione dello schema. Può avvenire in più fasi successive.

Definizione dei Dati : Le Tabelle

```
CREATE TABLE NomeTabella (  
    NomeAttributo Dominio [ ValoreDiDefault ] [Vincoli]  
    { NomeAttributo Dominio [ ValoreDiDefault ] [Vincoli] }  
    [AltriVincoli] )
```

Istruzione **CREATE TABLE**:

- Definisce uno schema di relazione e ne crea un'istanza vuota.
- Specifica gli attributi
- Per ogni attributo valore di default ed eventuali vincoli
- altri vincoli a livello di tabella (tra più attributi).

Definizione dei Dati : Le Tabelle

```
CREATE TABLE NomeTabella (  
    NomeAttributo Dominio [ ValoreDiDefault ] [Vincoli]  
    { NomeAttributo Dominio [ ValoreDiDefault ] [Vincoli] }  
    [AltriVincoli] )
```

- Una tabella è costituita da una collezione ordinata di attributi e da un insieme (eventualmente vuoto) di vincoli.
- Parleremo di tabelle e non di relazioni, e di righe e non di tuple, poichè rispetto al modello relazionale possiamo avere duplicazione di righe.

CREATE TABLE: Un esempio

```
CREATE TABLE Impiegato(  
  Matricola CHAR(6) PRIMARY KEY,  
  Nome CHAR(20) NOT NULL,  
  Cognome CHAR(20) NOT NULL,  
  Dipart CHAR(15),  
  Stipendio NUMERIC(9) DEFAULT 0,  
  FOREIGN KEY(Dipart) REFERENCES  
  Dipartimento(NomeDip),  
  UNIQUE (Cognome, Nome)  
)
```

Attributi con
rispettivi domini
e vincoli

Vincoli

Definizione dei Dati : I Domini

I domini specificano i valori ammessi di ciascun attributo

- Domini elementari (SQL ha 6 domini predefiniti)

1. Carattere
2. Numerico Esatto
3. Numerico Approssimato
4. Data/Ora
5. Intervallo Temporale
6. Bit (SQL-2)

Bit è stato poi eliminato/sostituito parzialmente da BOOLEAN in SQL-3

- Domini definiti dall'utente (semplici, ma riutilizzabili)

Domini elementari (predefiniti)

Testuali: e.g.: VARCHAR(n) stringa di lunghezza variabile tra 0 e n

Numerici: e.g.: INTEGER, SMALLINT, FLOAT,

Data e ora: e.g.: TIMESTAMP, DATE, TIME

Booleani: con dominio 0,1

BLOB, CLOB: oggetti di grandi dimensioni

- Costituiti da binari (Blob) o caratteri (Clob).
- Memorizzati e trattati diversamente dagli altri,
- Usati per informazioni non strutturate (testi) o multimediali (immagini, video).

Domini Elementari : Il tipo carattere

Rappresenta singoli caratteri alfanumerici oppure stringhe di lunghezza fissa o variabile

`character [varying] [(lunghezza)]`

`character` o `char`

`character(lunghezza)`

`varchar(lunghezza)`

`character varying (lunghezza)`

Si definisce l'attributo `Nome` della relazione `IMPIEGATI` come sequenza di caratteri di lunghezza massima 20

Nome **`char(20)`**

Nome **`varchar(20)`**

Paolo _ Bianchi _ _ _ _ _

Paolo _ Bianchi

Domini Elementari : I Tipi Numerici Esatti

Rappresentano numeri interi o numeri decimali in virgola fissa (con un numero prefissato di decimali, ad esempio i valori monetari). *Precision* è numero di cifre significative, *scala* il numero di cifre dopo la virgola.

integer
smallint

numeric **numeric**(*precisione*) **numeric**(*precisione*,*scala*)
decimal **decimal**(*precisione*) **decimal**(*precisione*,*scala*)

Si definisce l'attributo Eta nella relazione IMPIEGATI

Eta **decimal**(2) Rappresenta tutti i numeri fra -99 e +99

Si definisce l'attributo Cambio nella relazione PAGAMENTO per indicare il valore del cambio di una certa moneta preciso al centesimo

Cambio **numeric**(6,2) Rappresenta tutti i numeri fra -9999,99 e +9999,99

Domini Elementari : I Tipi Numerici Esatti

INTEGER / SMALLINT rappresentano valori interi.

La precisione (numero totale di cifre) varia a seconda della specifica implementazione di SQL, non è specificata nello standard.
SMALLINT richiede minore spazio di memorizzazione.

NUMERIC / DECIMAL rappresentano i valori decimali.

La differenza tra NUMERIC e DECIMAL è che il primo deve essere implementato esattamente con la precisione richiesta, mentre il secondo può avere una precisione maggiore, la precisione segnalata è requisito minimo.

Se la precisione non è specificata si usa il valore caratteristico dell'implementazione. Per la scala si usa valore 0.

Domini Elementari : Tipi numerici approssimati

Sono utili per rappresentare valori reali approssimati, ad esempio grandezze fisiche (rappresentazione in virgola mobile, in cui a ciascun numero corrisponde una coppia di valori: mantissa e esponente).

```
float                float(precisione)  
double precision  
real
```

Si definisce l'attributo `Massa` nella tabella ASTEROIDI

```
Massa real
```

`0,17E16` = `1,7` 10^{15}

mantissa esponente, num intero

Domini Elementari : Tipi numerici approssimati

REAL / DOUBLE PRECISION rappresentano valori a singola / doppia precisione in virgola mobile.

La precisione (lunghezza della mantissa) dipende dalla specifica implementazione di SQL.

FLOAT permette di richiedere la precisione che si desidera (lunghezza della mantissa). L'esponente dipende ancora dall'implementazione

Domini definiti dagli utenti

Partendo dai domini predefiniti è possibile costruire nuovi domini tramite la primitiva CREATE DOMAIN.

Un nuovo dominio è caratterizzato dalle seguenti informazioni: nome, dominio elementare, valore di default, insieme di vincoli (constraints)

```
CREATE DOMAIN NomeDominio AS DominioElementare  
    [ ValoreDefault ]  
    [ Constraints ]
```

L'Istruzione CREATE DOMAIN definisce un dominio (elementare), utilizzabile in definizioni di relazioni, anche con vincoli e valori di default.

CREATE DOMAIN - Esempio

```
CREATE DOMAIN Voto AS SMALLINT  
    DEFAULT 0  
    NOT NULL
```

Il nuovo dominio Voto è definito come uno SMALLINT con valore di default e che non deve essere null.

la definizione di “nuovi domini” è utile perché permette di associare dei vincoli a un nome di dominio: questo è importante quando si deve ripetere la stessa definizione di attributo su diverse tabelle: ad esempio, **modifiche** alla definizione di Voto si ripercuotono in tutte le occorrenze di questo dominio nello schema del Database.

CREATE DOMAIN - Esempio

```
CREATE DOMAIN Voto AS SMALLINT  
    DEFAULT 0  
    CHECK (value >= 18 AND value <= 30)
```

Posso mettere vincoli più o meno utili

Al contrario dei meccanismi di definizione dei tipi nei linguaggi di programmazione, SQL-2 non mette a disposizione dei costruttori di dominio come record o array.

Questa caratteristica deriva dal modello relazionale dei dati il quale richiede che ogni attributo sia definito su un dominio elementare.

CREATE TABLE - Esempio

```
CREATE TABLE NomeTabella (  
    NomeAttributo Dominio [ ValoreDiDefault ] [Vincoli]  
    { NomeAttributo Dominio [ ValoreDiDefault ] [Vincoli] }  
    [AltriVincoli] )
```

```
CREATE TABLE Impiegato (  
    Matricola CHAR(6) PRIMARY KEY,  
    Nome CHAR(20) NOT NULL,  
    Cognome CHAR(20) NOT NULL,  
    Dipart CHAR(15)  
    Stipendio NUMERIC(9) DEFAULT 0,  
    UNIQUE (Cognome, Nome)  
)
```

Valori di default per il dominio

Definiscono il valore che deve assumere l'attributo quando non viene specificato un valore durante l'inserimento di una tupla

DEFAULT < *ValoreGenerico* | user | null >

- *ValoreGenerico* rappresenta un valore compatibile con il dominio, rappresentato come una costante o come un'espressione.
- *user* è l'identificativo dell'utente che effettua il comando di aggiornamento della tabella.
- *null* è il valore di DEFAULT di base.

Definizione dei Dati : Il valore NULL

E' un valore polimorfico (che appartiene a tutti i domini) col significato di valore non noto:

- 1) il valore esiste in realtà ma è ignoto al database
(es.: data di nascita)
- 2) il valore è inapplicabile
(es.: numero patente per minorenni)
- 3) non si sa se il valore è inapplicabile o meno
(es.: numero patente per un maggiorenne)

Definizione dei Dati : Vincoli (Constraints)

Un vincolo è una regola che specifica delle condizioni sui valori di un elemento dello schema del database.

Un vincolo può essere associato ad una tabella, ad un attributo, ad un dominio.

- **Vincoli Intrarelazionali:** Proprietà sempre valida all'interno di una relazione
- **Vincoli Interrelazionali:** Proprietà sempre valida tra relazioni diverse

Vincoli Intrarelazionali

NOT NULL Il valore deve essere non nullo

UNIQUE I valori devono essere non ripetuti

PRIMARY KEY Chiave primaria (una sola per tabella)

CHECK Definisce condizioni complesse
(sia intra che inter -relazionali)

Vincoli Intrarelazionali : Esempio

```
CREATE TABLE Impiegato (
```

```
    Matricola    CHAR(6)    PRIMARY KEY,
```

```
    Nome        CHAR(20)   NOT NULL,
```

```
    Cognome     CHAR(20)   NOT NULL,
```

```
    Stipendio   NUMERIC(9) DEFAULT 0,
```

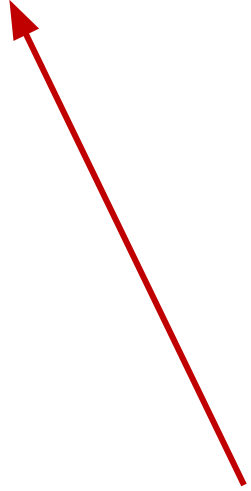
```
    UNIQUE (Cognome, Nome)
```

```
)
```

Vincolo su più attributi



Vincolo su un attributo



Vincoli Intrarelazionali : UNIQUE

CREATE TABLE Impiegato (

Matricola CHAR(6) PRIMARY KEY,

Nome CHAR(20) NOT NULL,

Cognome CHAR(20) NOT NULL,

Stipendio NUMERIC(9) DEFAULT 0,

UNIQUE (Cognome, Nome)

)

Un attributo: Cognome CHAR(20) NOT NULL

Un insieme di attributi: UNIQUE (Cognome, Nome)

Con il vincolo UNIQUE **non possono avere istanze uguali ripetute.**

□ l'attributo o attributi sono (super)chiave

Chiave

- insieme di attributi che identificano univocamente le tuple di una relazione

Formalmente:

- un insieme K di attributi è **superchiave** per r se r non contiene due tuple distinte t_1 e t_2 con $t_1[K] = t_2[K]$
- K è **chiave** per r se è una superchiave minimale per r (cioé non contiene un'altra superchiave)

Vincoli Intrarelazionali : UNIQUE

CREATE TABLE Impiegato (

Matricola CHAR(6) PRIMARY KEY,

Nome CHAR(20) NOT NULL,

Cognome CHAR(20) NOT NULL,

Stipendio NUMERIC(9) DEFAULT 0,

UNIQUE (Cognome, Nome)

)

eccezione: valore null può comparire su diverse righe senza violare il vincolo.

Si assume che i valori null siano tutti diversi in diverse occorrenze.

Vincoli Intrarelazionali : UNIQUE

```
CREATE TABLE Impiegato (  
  Matricola    CHAR(6)    PRIMARY KEY,  
  Nome         CHAR(20)   NOT NULL,  
  Cognome      CHAR(20)   NOT NULL,  
  Stipendio    NUMERIC(9) DEFAULT 0,  
  UNIQUE (Cognome, Nome)  
)
```

≠

NON hanno lo
stesso effetto

```
CREATE TABLE Impiegato (  
  Matricola    CHAR(6)    PRIMARY KEY,  
  Nome         CHAR(20)   NOT NULL UNIQUE,  
  Cognome      CHAR(20)   NOT NULL UNIQUE,  
  Stipendio    NUMERIC(9) DEFAULT 0,  
)
```

Nome	Cognome
Luca	Rossi
Giovanni	Bianchi
Emilio	Verdi
Emilio	Rossi

Nome	Cognome
Luca	Neri
Giovanni	Bianchi
Emilio	Verdi
Arianna	Rossi

Vincoli Intrarelazionali: PRIMARY KEY

CREATE TABLE Impiegato (

Matricola CHAR(6) PRIMARY KEY,

Nome CHAR(20) NOT NULL,

Cognome CHAR(20) NOT NULL,

Stipendio NUMERIC(9) DEFAULT 0,

UNIQUE (Cognome, Nome)

)

Il vincolo PRIMARY KEY può essere definito una sola volta all'interno della relazione.

Definisce la chiave primaria (UNIQUE+NOT NULL)

Vincoli Intrarelazionali: PRIMARY KEY

```
CREATE TABLE Impiegato (  
    Nome      CHAR(20) NOT NULL,  
    Cognome   CHAR(20) NOT NULL,  
    Stipendio NUMERIC(9) DEFAULT 0,  
    PRIMARY KEY(Cognome, Nome)  
)
```

Può essere espresso da uno o più attributi

ATTENZIONE: In alcune implementazioni di SQL potrebbe essere necessario specificare comunque anche il vincolo NOT NULL per tutti gli attributi coinvolti.

Vincoli Intrarelazionali: CHECK

Introdotta in SQL-2, permette di definire vincoli più complessi di quelli standard (sia intra che inter-relazionali).

```
CREATE DOMAIN Voto AS SMALLINT  
    DEFAULT 0  
    NOT NULL
```

```
CREATE DOMAIN Voto AS SMALLINT  
    DEFAULT 0  
    CHECK (Voto >= 18 AND VOTO <= 30)
```